Handling Data Using React State

Introduction to State Management in React: Building a To-Do Application

Let's continue building our application. Previously, we discussed how we might organize data, such as a list of tasks, or "todos," within our React project. Now, we will delve into the process of dynamically updating and rendering components using something called state. This concept is fundamental to React, so pay close attention.

To-Do Structure Overview

Let's begin by reviewing our current ‘to-do’ structure. We have a list with four properties:

1. Feed the puppy
2. Water the plants
3. Make dinner
4. Charge the phone battery

If we click "add new to-do," we get a new element added to our list. However, clicking this button doesn't seem to change anything visually, other than the to-do item being added to the very end of the list. What we really want is for the table to re-render and show the new changes.

Introduction to State

This is where the concept of state comes into play. State is not specific to React, but React heavily depends on each component handling its own state. By manipulating the state, we can re-render our React components on the fly for the user.

Implementing useState Hook

Now, how do we bring state into our project? Here's the process:

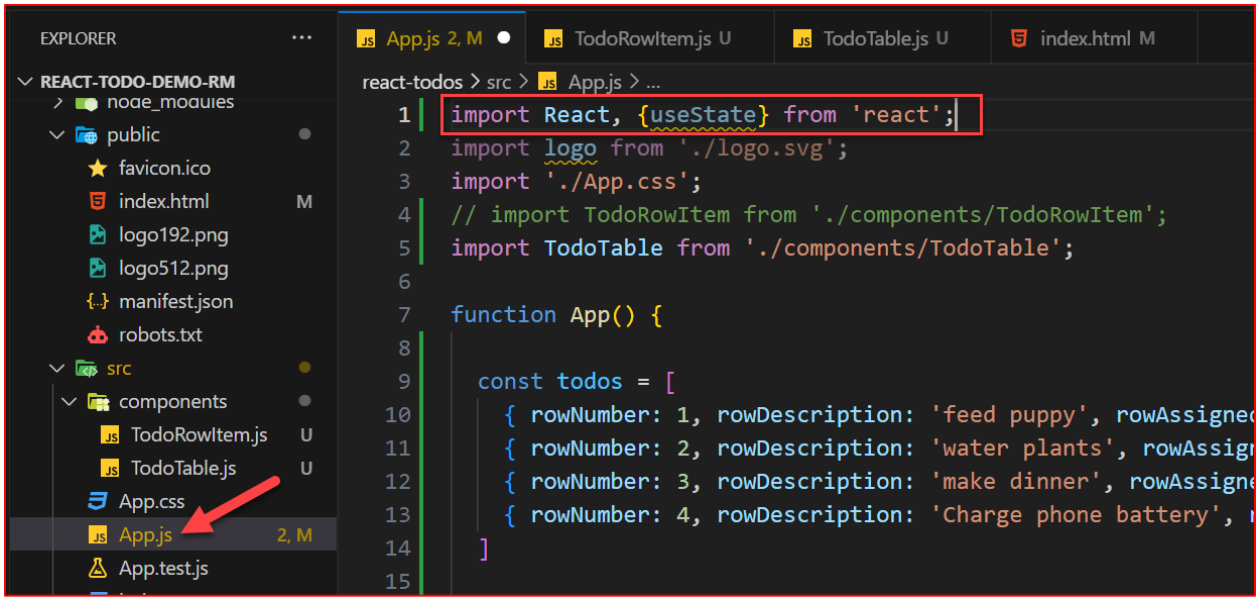
1. **Import React:** Start by importing React itself with the line

   ```
   import React from 'react';
   ```

2. **Use the useState Hook:** Next, we'll need to import the useState hook by adding

   ```
   {useState}
   ```

   after importing React.



The useState hook is a critical tool in React that returns a stateful value and a function to update that value. The term "use" in front of something we're importing indicates a React hook. There are other important hooks in React, but for now, we'll just focus on useState.

Applying useState to Our ToDos

To use useState, we'll replace our current to-dos with a useState variable. We'll also wrap the useState call around all of our to-do items. It returns two items in an array:

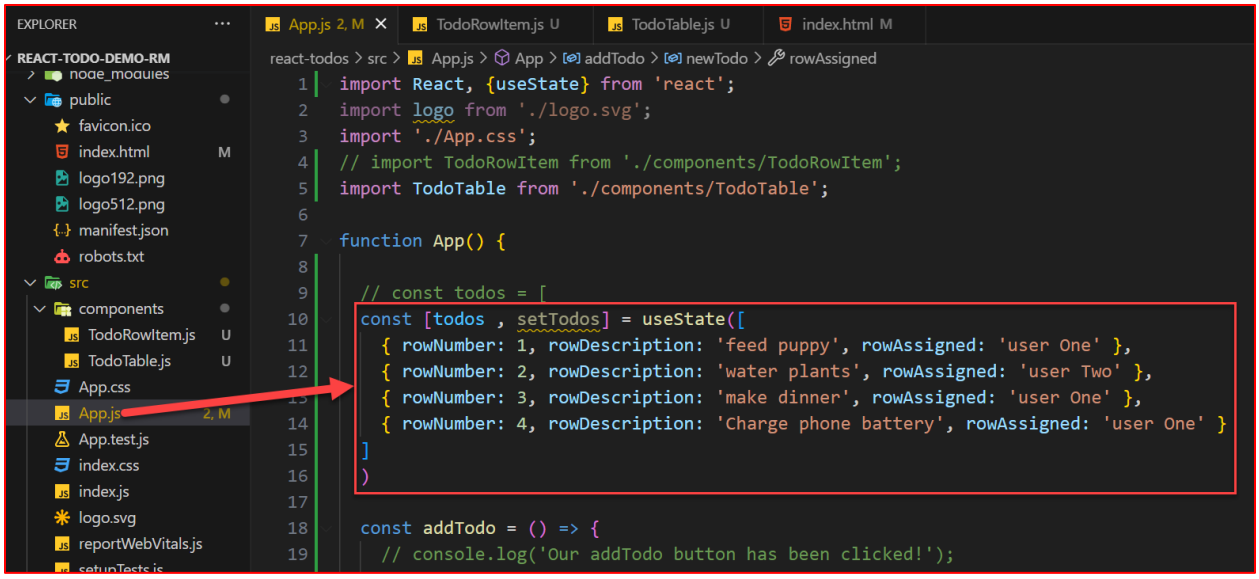
- The first element is the variable that uses the state, allowing us to access the to-dos.
- The second is a function to update that state.

Creating a Constant Variable with useState

The most straightforward way to use useState to manage this data is to create a constant variable right in front of it, using array destructuring. We'll declare it like this:

```
const [todos, setTodos] = useState();
```

Here, `todos` references all the information inside useState, and `setTodos` is used to update the `todos` variable. You can name these whatever you prefer, but the standard in React is to name them as above.



Refreshing the Application

If we now move forward and save the application, refreshing our browser will initially show the exact same data. That's because we're still using the `todos` variable passed as references within our application.

When we click to add to our to-dos, it appears to work, but we don't see any change in the state. This is intriguing!

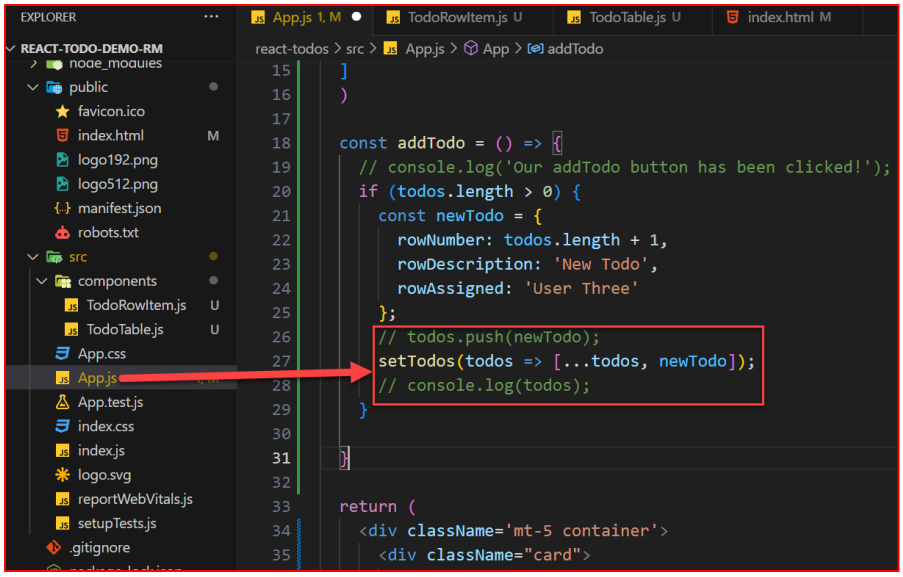
Using setTodos to Change the State

This is happening because we're not using the `setTodos` function to change the state, which is what re-renders the page. If we don't use `setTodos`, there will be no re-rendering.

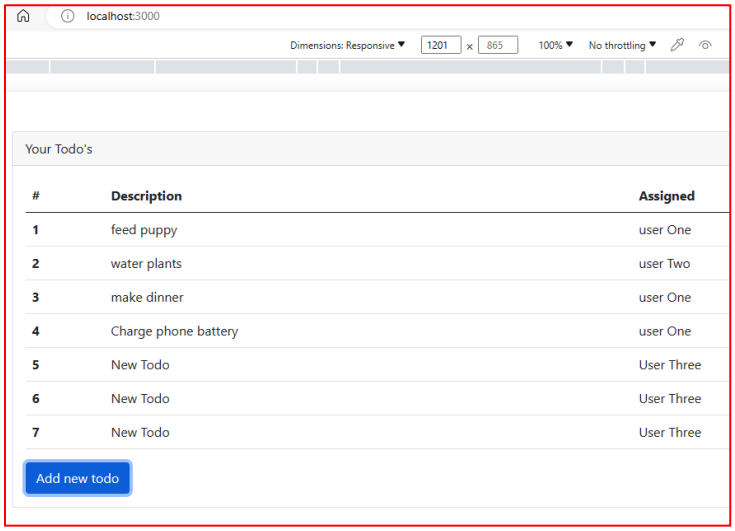
To fix this, instead of pushing a new todo, we'll use `setTodos`. Inside `setTodos`, we'll set the current to-dos that we want to be re-rendered, destructure the existing to-dos, and add our new to-do:

```
setTodos(todos => [...todos, newTodo]);[1]
```

[1] The spread operator (...) is used to create a shallow copy of an array (or an object) and can insert new items into the array (or new key-value pairs into the object). In this case, `...todos` takes all the items from the todos array and spreads them into a new array. The function `setTodos(todos => [...todos, newTodo]);` is updating the state of a todos array by appending a newTodo item to the end of the array. Instead of modifying the original todos array (which would be against React's principle of immutability for state), it creates a new array with all the original items and adds the newTodo to the end of this new array. The new array then becomes the updated state.



After saving and refreshing, we'll now see that when we click "add new todo," it automatically renders the new item, re-adding it to our application.



Recap

So, what did we achieve?

- We introduced the concept of state and the `useState` hook in React.
- We replaced our standard todo variable with stateful variables using `useState`.
- We implemented `onClick` listeners to call our "`addTodo`" function and updated our to-do list using the `setTodos` function.

This is fantastic, and understanding how to dynamically re-render the page is a critical skill in React. We've seen how we can update the UI dynamically for the user or client.